

7

Rediscovering Fire – Common Game Design Elements

Video games are getting more and more intricate every day. It seems that innovative ideas are on the rise, especially with the increasing popularity of indie games, such as *Minecraft* and *Super Meat Boy*. While the game ideas themselves are getting more and more abstract, at least on the outside, the rigid skeleton behind the pretty skin that keeps it standing and helps it retain shape is still taking the place of the lowest common denominator in the eyes of game developers. Even if the focus of the game centers around two unicorns who spend their free time smoking fairy dust and helping Dracula make muffins so that Neptune doesn't blow up, that concept coming to life is going to depend greatly on the underlying logic of the game before anything else. If there are no entities in the game, there are no unicorns. If the entities are simply bouncing around a black screen, the game is not engaging. These are the most common game design elements that any project must be able to fall back on, otherwise it is doomed to fail.

In this chapter, we will be covering the following:

- Designing and implementing the game map class
- Populating the map by creating and managing entities
- Checking for and handling collisions
- Meshing all of our code together into a finished game

The game map

The actual environment and surroundings a player explores are just as important as the rest of the game. Without the world being present, the player is simply left spinning in an empty void of the screen clear color. Designing a good interface to bring out various parts of the game, ranging from the level backdrop to numerous hazards our player has to face can be tricky. Let's build a solid foundation for that right now, starting with defining what our map format is going to be like, as we take a look ahead to determine what we want to accomplish:



First, we want to specify a texture handle as the background. Then, we want to clearly define the map size and set up the gravity, which determines how fast entities fall to the ground. Additionally, we need to store the default friction, which determines how slippery the average tile is. The last property we want to store is the name of the next map that gets loaded when the end of the current map is reached. Here is a snippet from one of the maps that we will be working with, `Map1.map`:

```
|type|~id|x|y|
BACKGROUND Bg1
SIZE 63 32
GRAVITY 512
DEFAULT_FRICTION 0.8 0
NEXTMAP map2.map
```

```
|PLAYER 0 512
|ENEMY Rat 128 512
TILE 0 0 25
TILE 1 0 26 WARP
...
```

As you can tell, in addition to defining all of the things discussed, the map file also stores the player position, as well as different enemies and their spawn positions. The last but definitely not the least important part of it is tile storage and the indication of which tile is going to "warp" the player to the next stage when touched.

What is a tile?

The term "tile" keeps getting thrown around, but it hasn't been defined yet. To put it simply, a tile is one of the many segments that make up the world. Tiles are blocks that create the game environment, whether it's the grass you're standing on or the spikes you're falling onto. The map uses a tile sheet, which is fairly similar to a sprite sheet, in that it holds many different sprites at once. The main difference is how those sprites are obtained from the tile sheet. This is what the texture that is going to be used as a tile sheet looks like in our case:



Each tile also has unique properties, which we will want to load from the `Tiles.cfg` file:

```
|id|name|friction x|friction y|deadly
0 Grass 0.8 0 0
1 Dirt 0.8 0 0
2 Stone 0.8 0 0
3 Brick 0.8 0 0
4 Brick_Red 0.8 0 0
5 Rock 0.8 0 0
6 Icy_Rock 0.6 0 0
7 Spikes 1.0 0 1
8 Ice 0.25 0 0
```

It is quite simple and only contains the tile ID, name, both axes of friction, and a binary flag for the tile being deadly to touch.

Building the game world

Since tiles are going to play such a huge role in our game design, it would be greatly helpful to have a separate data structure that all tile information can be localized to. A good place to start is by defining some constants of the tile size, as well as dimensions of the tile sheets that are going to be used. A simple enumeration can be quite helpful when storing this information:

```
enum Sheet{Tile_Size = 32, Sheet_Width = 256, Sheet_Height = 256};
```

Here, we make it so all tiles are going to be 32 px wide and 32 px tall and every single tile sheet is going to be 256 px wide and 256 px tall. These constants, obviously, can be changed, but the idea here is to keep them the same during runtime.

To keep our code a little shorter, we can also benefit from a type alias for tile IDs:

```
using TileID = unsigned int;
```

The flyweight pattern

Each tile, obviously, has to have a sprite that represents its type graphically speaking. In order to draw a grass tile, we want to adjust the sprite to be cropped to only the grass tile in the tile sheet. Then, we set its position on the screen and draw it. Seems simple enough, but consider the following situation: you have a map that's 1000x1000 tiles in size, and perhaps 25% of that map's size is actual tiles that aren't just air, which leaves you with the total amount of 62,500 tiles to draw. Now imagine you're storing a sprite with each tile. Granted, sprites are lightweight objects, but that's still a huge waste of resources. This is where the flyweight pattern comes in.

Storing huge chunks of redundant data is obviously a waste, so why not just store one instance of each type and simply store a pointer to the type in the tile? That, in a nutshell, is the flyweight pattern. Let's see it in action, by implementing a tile information structure:

```
struct TileInfo{
    TileInfo(SharedContext* l_context,
        const std::string& l_texture = "", TileID l_id = 0)
        : m_context(l_context), m_id(0), m_deadly(false)
    {
        TextureManager* tmgr = l_context->m_textureManager;
        if (l_texture == ""){ m_id = l_id; return; }
        if (!tmgr->RequireResource(l_texture)){ return; }
        m_texture = l_texture;
        m_id = l_id;
        m_sprite.setTexture(*tmgr->GetResource(m_texture));
        sf::IntRect tileBoundaries(m_id %
```

```

        (Sheet::Sheet_Width / Sheet::Tile_Size) * Sheet::Tile_Size,
        m_id / (Sheet::Sheet_Height / Sheet::Tile_Size) * Sheet::Tile_Size,
        Sheet::Tile_Size, Sheet::Tile_Size);
    m_sprite.setTextureRect(tileBoundaries);
}

~TileInfo(){
    if (m_texture == ""){ return; }
    m_context->m_textureManager->ReleaseResource(m_texture);
}

sf::Sprite m_sprite;

TileID m_id;
std::string m_name;
sf::Vector2f m_friction;
bool m_deadly;

SharedContext* m_context;
std::string m_texture;
};

```

This struct essentially holds everything about every tile type that isn't unique. It stores the texture that it's using, as well as the sprite that will represent the tile. As you can see, in the constructor of this structure, we set the sprite to point to the tile sheet texture and then crop it based on its tile ID. This type of cropping is a little different than the one in the sprite sheet class, because now we only have the tile ID to work with, and we don't know which row the sprite is on. Using some basic math allows us to first figure out how many columns and rows the tile sheet has, by dividing our sheet dimensions by the tile size. In this case, a 256x256 px sized sprite sheet with tiles of 32x32 px in size would have eight tiles per row and column. Obtaining the coordinate of a tile ID on an x axis can be done by using the modulus operator `%`. In a case of eight tiles per row, it would return values from 0 to 7, based on the ID. Figuring out the y coordinate is done simply by dividing the ID by the number of tiles per column. This gives us the top-left coordinate of the tile sprite in the tile sheet, so we finish the cropping by passing in the `Sheet::Tile_Size`.

The `TileInfo` destructor simply frees the texture used for the tile sheet. The rest of the values stored in this structure will be initialized when the map is loaded. Now let's define our tile structure:

```

struct Tile{
    TileInfo* m_properties;
    bool m_warp; // Is the tile a warp.
    // Other flags unique to each tile.
};

```

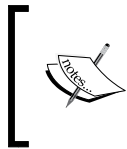
This is the reason why the flyweight pattern is so powerful. The tile objects are incredibly lightweight, if they only store information that can be unique to each tile and not the tile type. The only flag we're interested in so far is if the tile is a warp, which means it loads the next level when the player is standing on it.

Designing the map class

With tiles out of the way, we can move on to higher-level structures, such as the game map. Let's begin by creating a few suitable types of containers that will hold the map information, as well as the tile type information:

```
using TileMap = std::unordered_map<TileID, Tile*>;
using TileSet = std::unordered_map<TileID, TileInfo*>;
```

The `TileMap` type is an `unordered_map` container, which holds pointers to `Tile` objects that are addressed by an unsigned integer.



In cases where tile counts are known in advance, it would be prudent to use a container that will not change in size (such as `std::array` or a pre-allocated `std::vector`) in order to achieve continuous storage, and in turn, much faster access.

But wait a minute! Aren't we working in two dimensions? How are we mapping the tiles to only one integer, if the coordinates are represented by two numbers? Well, with a little bit of mathematics, it's entirely possible to manipulate indices of two dimensions to be represented as a single number. This will be covered shortly.

The `TileSet` data type represents the container of all different types of tiles, which are tied to a tile ID that's represented by the unsigned integer. This brings us everything we need in order to write the map header file, which might look a little something like this:

```
class Map{
public:
    Map(SharedContext* l_context, BaseState* l_currentState);
    ~Map();
    Tile* GetTile(unsigned int l_x, unsigned int l_y);
    TileInfo* GetDefaultTile();
    float GetGravity() const;
    unsigned int GetTileSize() const;
    const sf::Vector2u& GetMapSize() const;
    const sf::Vector2f& GetPlayerStart() const;
    void LoadMap(const std::string& l_path);
    void LoadNext();
    void Update(float l_dT);
    void Draw();
```

```
private:
    // Method for converting 2D coordinates to 1D ints.
    unsigned int ConvertCoords(unsigned int l_x, unsigned int l_y);
    void LoadTiles(const std::string& l_path);
    void PurgeMap();
    void PurgeTileSet();

    TileSet m_tileSet;
    TileMap m_tileMap;
    sf::Sprite m_background;
    TileInfo m_defaultTile;
    sf::Vector2u m_maxMapSize;
    sf::Vector2f m_playerStart;
    unsigned int m_tileCount;
    unsigned int m_tileSetCount;
    float m_mapGravity;
    std::string m_nextMap;
    bool m_loadNextMap;
    std::string m_backgroundTexture;
    BaseState* m_currentState;
    SharedContext* m_context;
};
```

First, we define all the predictable methods, such as obtaining a tile at specific coordinates, getting various information from the class, and, of course, methods for updating and drawing the map. Let's move on to the implementation of these methods, in order to talk about them more in depth:

```
Map::Map(SharedContext* l_context, BaseState* l_currentState)
    :m_context(l_context), m_defaultTile(l_context),
      m_maxMapSize(32, 32), m_tileCount(0), m_tileSetCount(0),
      m_mapGravity(512.f), m_loadNextMap(false),
      m_currentState(l_currentState)
{
    m_context->m_gameMap = this;
    LoadTiles("tiles.cfg");
}
```

The map constructor initializes its data members to some default values and calls a private method in order to load different types of tiles from the `tiles.cfg` file. Fairly standard. Predictably enough, the destructor of this class does nothing out of the ordinary either:

```
Map::~Map() {
    PurgeMap();
}
```

```
PurgeTileSet();  
m_context->m_gameMap = nullptr;  
}
```

Obtaining tiles from the map is done by first converting the 2D coordinates provided as arguments to this method into a single number, and then locating the specific tile in an unordered map:

```
Tile* Map::GetTile(unsigned int l_x, unsigned int l_y){  
    auto itr = m_tileMap.find(ConvertCoords(l_x,l_y));  
    return(itr != m_tileMap.end() ? itr->second : nullptr);  
}
```

The conversion of coordinates looks like this:

```
unsigned int Map::ConvertCoords(const unsigned int& l_x,  
    const unsigned int& l_y)  
{  
    return (l_x * m_maxMapSize.x) + l_y; // Row-major.  
}
```

In order for this to work, we must have the maximum size of the map defined, otherwise it will produce wrong results.

Updating the map is another crucial part:

```
void Map::Update(float l_dT){  
    if(m_loadNextMap){  
        PurgeMap();  
        m_loadNextMap = false;  
        if(m_nextMap != ""){  
            LoadMap("media/maps/"+m_nextMap);  
        } else {  
            m_currentState->GetStateManager()->  
                SwitchTo(StateType::GameOver);  
        }  
        m_nextMap = "";  
    }  
    sf::FloatRect viewSpace = m_context->m_wind->GetViewSpace();  
    m_background.setPosition(viewSpace.left, viewSpace.top);  
}
```

Here, it checks the `m_loadNextMap` flag. If it's set to `true`, the map information gets purged and the next map is loaded, if the data member that holds its handle is set. If it isn't, the application state is set to `GameOver`, which will be created later. This will simulate the player beating the game. Finally, we obtain the view space of the window and set our map background's top-left corner to the view space's left corner in order for the background to follow the camera. Let's draw these changes on the screen:

```
void Map::Draw() {
    sf::RenderWindow* l_wind = m_context->m_wind->GetRenderWindow();
    l_wind->draw(m_background);
    sf::FloatRect viewSpace = m_context->m_wind->GetViewSpace();

    sf::Vector2i tileBegin(
        floor(viewSpace.left / Sheet::Tile_Size),
        floor(viewSpace.top / Sheet::Tile_Size));
    sf::Vector2i tileEnd(
        ceil((viewSpace.left + viewSpace.width) / Sheet::Tile_Size),
        ceil((viewSpace.top + viewSpace.height) / Sheet::Tile_Size));

    unsigned int count = 0;
    for(int x = tileBegin.x; x <= tileEnd.x; ++x) {
        for(int y = tileBegin.y; y <= tileEnd.y; ++y) {
            if(x < 0 || y < 0) { continue; }
            Tile* tile = GetTile(x,y);
            if (!tile) { continue; }
            sf::Sprite& sprite = tile->m_properties->m_sprite;
            sprite.setPosition(x * Sheet::Tile_Size,
                               y * Sheet::Tile_Size);
            l_wind->draw(sprite);
            ++count;
        }
    }
}
```

A pointer to the render window is obtained through the share context and the background is drawn in the first two lines here. The next three lines serve a purpose, simply known by a name of culling. It is a technique that any good game programmer should utilize, where anything that's not currently within the view space of the screen should be left undrawn. Once again, consider the situation where you have a massive map of size 1000x1000. Although modern hardware nowadays could draw that really fast, there's still no need to waste those clock-cycles when they could instead be used to perform a much better task, instead of bringing something to the screen that isn't even visible. If you are not culling anything in your game, it will eventually start taking serious performance hits.

The tile coordinates all the way from the top-left corner of the view space to its bottom-right corner are fed into a loop. First, they get evaluated to be positive. If they're negative, the way we calculate our 1D index for the map container will produce some mirroring artifacts, where the same map you see will be repeated over and over again if you go up or left far enough.

A pointer to a tile is obtained by passing in the x and y coordinates from the loop. If it is a valid tile, we obtain its sprite from the pointer to the `TileInfo` structure. The position of the sprite is set to match the coordinates of the tile and the sprite is drawn on screen.

Now for a way to erase the entire map:

```
void Map::PurgeMap() {
    m_tileCount = 0;
    for (auto &itr : m_tileMap) {
        delete itr.second;
    }
    m_tileMap.clear();
    m_context->m_entityManager->Purge();

    if (m_backgroundTexture == "") { return; }
    m_context->m_textureManager->
        ReleaseResource(m_backgroundTexture);
    m_backgroundTexture = "";
}
```

In addition to clearing the map container, you will notice that we're calling the `Purge` method of an entity manager. For now, ignore that line. Entities will be covered shortly. We must also not forget to free up the background texture when erasing the map.

Emptying the container of different tile types is also a necessary part:

```
void Map::PurgeTileSet() {
    for (auto &itr : m_tileSet) {
        delete itr.second;
    }
    m_tileSet.clear();
    m_tileSetCount = 0;
}
```

This will most likely only be called in the destructor, but it's still nice to have a separate method. Speaking of different tile types, we need to load them in from a file:

```
void Map::LoadTiles(const std::string& l_path){
    std::ifstream file;
    file.open(Utils::GetWorkingDirectory() + l_path);
    if (!file.is_open()){
        std::cout << "! Failed loading tile set file: "
            << l_path << std::endl;
        return;
    }
    std::string line;
    while(std::getline(file,line)){
        if (line[0] == '|'){ continue; }
        std::stringstream keystream(line);
        int tileId;
        keystream >> tileId;
        if (tileId < 0){ continue; }
        TileInfo* tile = new TileInfo(m_context,"TileSheet",tileId);
        keystream >> tile->m_name >> tile->m_friction.x
            >> tile->m_friction.y >> tile->m_deadly;
        if(!m_tileSet.emplace(tileId,tile).second){
            // Duplicate tile detected!
            std::cout << "! Duplicate tile type: "
                << tile->m_name << std::endl;
            delete tile;
        }
    }
    file.close();
}
```

The tile ID gets loaded first, as the `tiles.cfg` format suggests. It gets checked for being out of bounds, and if it isn't, dynamic memory is allocated for the tile type, at which point all of its internal data members are initialized to the values from the string stream. If the tile information object cannot be inserted into the tile set container, there must be a duplicate entry, and the dynamic memory is de-allocated.

Now for the grand finale of the map – the loading method. Since the actual file loading code remains pretty much the same, let's jump right to reading the contents of the map file, starting with tile entries:

```
if(type == "TILE"){
    int tileId = 0;
    keystream >> tileId;
    if (tileId < 0){ std::cout << "! Bad tile id: "
        << tileId << std::endl;
```

```
        continue;
    }
    auto itr = m_tileSet.find(tileId);
    if (itr == m_tileSet.end()){
        std::cout << "! Tile id(" << tileId
            << ") was not found in tileset." << std::endl;
        continue;
    }
    sf::Vector2i tileCoords;
    keystream >> tileCoords.x >> tileCoords.y;
    if (tileCoords.x>m_maxMapSize.x || tileCoords.y>m_maxMapSize.y)
    {
        std::cout << "! Tile is out of range: " <<
            tileCoords.x << " " << tileCoords.y << std::endl;
        continue;
    }
    Tile* tile = new Tile();
    // Bind properties of a tile from a set.
    tile->m_properties = itr->second;
    if(!m_tileMap.emplace(ConvertCoords(
        tileCoords.x,tileCoords.y),tile).second)
    {
        // Duplicate tile detected!
        std::cout << "! Duplicate tile! : " << tileCoords.x
            << " " << tileCoords.y << std::endl;
        delete tile;
        tile = nullptr;
        continue;
    }
    std::string warp;
    keystream >> warp;
    tile->m_warp = false;
    if(warp == "WARP"){ tile->m_warp = true; }
} else if ...
```

The first segment of the `TILE` line is loaded in, which is the tile ID. It is checked, as per usual, to be within the boundaries of positive numbers and 0. If it is, the tile information of that specific tile ID is looked up in the tile set. Because we don't want empty tiles around our map, we only proceed if the tile information of the specific ID is located. Next, the tile coordinates are read in and checked for being within the boundaries of the map size. If they are, the memory for the tile is allocated and its tile information data member is set to point to the one located in the tile set. Lastly, we attempt to read in a string at the end of the `TILE` line and check if it says "WARP". That's the indication that touching a specific tile should load the next level.

Now for the background of the map:

```

    } else if (type == "BACKGROUND") {
        if (m_backgroundTexture != "") { continue; }
        keystream >> m_backgroundTexture;
        if (!m_context->m_textureManager->
            RequireResource(m_backgroundTexture))
        {
            m_backgroundTexture = "";
            continue;
        }
        sf::Texture* texture = m_context->m_textureManager->
            GetResource(m_backgroundTexture);
        m_background.setTexture(*texture);
        sf::Vector2f viewSize = m_currentState->GetView().getSize();
        sf::Vector2u textureSize = texture->getSize();
        sf::Vector2f scaleFactors;
        scaleFactors.x = viewSize.x / textureSize.x;
        scaleFactors.y = viewSize.y / textureSize.y;
        m_background.setScale(scaleFactors);
    } else if ...

```

This one is quite straightforward. A texture handle gets loaded from the BACKGROUND line. If the handle is valid, the background sprite gets tied to the texture. There is a catch though. Let's say that the view of our window is larger than the texture of the background. That would result in empty areas all around the background, which looks horrendous. Repeating the texture might remedy the empty areas, but the specific backgrounds we're going to be working with don't tile well, so the best solution is to scale the sprite enough to fit the view space fully, whether it's larger or smaller. The factors of the scaling can be obtained by multiplying the size of the view by the size of the texture. If, for example, we have a view that's 800x600 px large and a texture of a size 400x300 px, the scale factor for both axes would be 2 and the background is scaled up to twice its size.

Next is the easy part of simply reading in some data members from a file:

```

    } else if (type == "SIZE") {
        keystream >> m_maxMapSize.x >> m_maxMapSize.y;
    } else if (type == "GRAVITY") {
        keystream >> m_mapGravity;
    } else if (type == "DEFAULT_FRICTION") {
        keystream >> m_defaultTile->m_friction.x
            >> m_defaultTile->m_friction.y;
    } else if (type == "NEXTMAP") {
        keystream >> m_nextMap;
    }

```

Let's wrap this class up with a little helper method that will help us keep track of when the next map should be loaded:

```
void Map::LoadNext() { m_loadNextMap = true; }
```

This concludes the map class implementation. The world now exists, but nobody is there to occupy it. Outrageous! Let's not insult our work and create some entities to explore the environments we conjure up.

The parent of all world objects

An entity is essentially just another word for a game object. It's an abstract class that acts as a parent to all of its derivatives, which include the player, enemies, and perhaps even items, depending on how you want to implement that. Having these entirely different concepts share the same roots allows the programmer to define types of behavior that are common to all of them. Moreover, it lets the game engine act upon them in the same manner, as they all share the same interface. For example, the enemy can be pushed, and so can the player. All enemies, items, and the player have to be affected by gravity as well. Having that common ancestry between these different types allows us to offload a lot of redundant code and focus on the aspects that are unique to each entity, instead of re-writing the same code over and over again.

Let's begin by defining what entity types we're going to be dealing with:

```
enum class EntityType{ Base, Enemy, Player };
```

The base entity type is just the abstract class, which will not actually be instantiated. That leaves us with enemies and a player. Now to set up all the possible states an entity can have:

```
enum class EntityState{  
    Idle, Walking, Jumping, Attacking, Hurt, Dying  
};
```

You have probably noticed that these states vaguely match the animations from the player sprite sheet. All character entities will be modeled this way.

Creating the base entity class

In cases where entities are built using inheritance, writing a basic parent class like this is fairly common. It has to provide any and all functionality that any given entity within the game should have.

With all of the setting up out of the way, we can finally start shaping it like so:

```
class EntityManager;
class EntityBase{
friend class EntityManager;
public:
    EntityBase(EntityManager* l_entityMgr);
    virtual ~EntityBase();
    ... // Getters and setters.
    void Move(float l_x, float l_y);
    void AddVelocity(float l_x, float l_y);
    void Accelerate(float l_x, float l_y);
    void SetAcceleration(float l_x, float l_y);
    void ApplyFriction(float l_x, float l_y);
    virtual void Update(float l_dT);
    virtual void Draw(sf::RenderWindow* l_wind) = 0;
protected:
    // Methods.
    void UpdateAABB();
    void CheckCollisions();
    void ResolveCollisions();
    // Method for what THIS entity does TO the l_collider entity.
    virtual void OnEntityCollision(EntityBase* l_collider,
        bool l_attack) = 0;
    // Data members.
    std::string m_name;
    EntityType m_type;
    unsigned int m_id; // Entity id in the entity manager.
    sf::Vector2f m_position; // Current position.
    sf::Vector2f m_positionOld; // Position before entity moved.
    sf::Vector2f m_velocity; // Current velocity.
    sf::Vector2f m_maxVelocity; // Maximum velocity.
    sf::Vector2f m_speed; // Value of acceleration.
    sf::Vector2f m_acceleration; // Current acceleration.
    sf::Vector2f m_friction; // Default friction value.
    TileInfo* m_referenceTile; // Tile underneath entity.
    sf::Vector2f m_size; // Size of the collision box.
    sf::FloatRect m_AABB; // The bounding box for collisions.
    EntityState m_state; // Current entity state.
    // Flags for remembering axis collisions.
    bool m_collidingOnX;
    bool m_collidingOnY;

    Collisions m_collisions;
    EntityManager* m_entityManager;
};
```

Right off the bat, we set up the `EntityManager` class that we haven't written yet to be a friend class of the base entities. Because the code might be a little confusing, a barrage of comments was added to explain every data member of the class, so we're not going to touch on those too much until we encounter them during the implementation of the class.

The three major properties of an entity include its position, velocity, and acceleration. The position of an entity is self explanatory. Its velocity represents how fast an entity is moving. Because all of the update methods in our application take in the delta time in seconds, the velocity is going to represent the number of pixels that an entity moves across per second. The last element of the major three is acceleration, which is responsible for how fast the entity's velocity increases. It, too, is defined as the number of pixels per second that get added to the entity's velocity. The sequence of events here is as follows:

1. The entity is accelerated and its acceleration adjusts its velocity.
2. The entity's position is re-calculated based on its velocity.
3. The velocity of an entity is damped by the friction coefficient.

Collisions and bounding boxes

Before jumping into implementations, let's talk about one of the most commonly used elements in all games – collisions. Detecting and resolving a collision is what keeps the player from falling through the map or going outside the screen. It's also what determines if a player gets hurt if they get touched by the enemy. In a round-about way, we used a basic form of collision detection in order to determine which tiles we should render in the map class. How does one detect and resolve collisions? There are many ways to do so, but for our purposes, the most basic form of a bounding box collision will do just fine. Other types of collisions that incorporate different shapes, such as circles, can also be used, but may not be the most efficient or appropriate depending on the kind of game that's being built.

A bounding box, much like it sounds, is a box or a rectangle which represents the solid portion of an entity. Here's a good example of a bounding box:



It isn't visible like that, unless we create an actual `sf::RectangleShape` with the same position and size as the bounding box and render that, which is a useful way to debug your applications. In our base entity class, the bounding box named `m_AABB` is simply a `sf::FloatRect` type. The name "AABB" represents two pairs of different values it holds: the position and the size. Bounding box collision, also referred to as an AABB collision, is simply a situation where two bounding boxes intersect with one another. The rectangle data types in SFML provide us with a method that checks for intersections:

```
if (m_AABB.intersects(SomeRectangle)) { ... }
```

The term collision resolution simply means performing some sequence of actions in order to notify and move the colliding entities. In a case of collision with tiles, for example, the collision resolution means pushing the entity back just far enough so it isn't intersecting with the tile any more.



The code files of this project contain an additional class that allows debug information rendering to take place, as well as all of these bits of information already set up. Hitting the `O` key will toggle its visibility.

Implementing the base entity class

With all of that information out of the way, we can finally return to implementing the base entity class. As always, what better place is there to start than the constructor? Let's take a look:

```
EntityBase::EntityBase(EntityManager* l_entityMgr)
    :m_entityManager(l_entityMgr), m_name("BaseEntity"),
    m_type(EntityType::Base), m_referenceTile(nullptr),
    m_state(EntityState::Idle), m_id(0),
    m_collidingOnX(false), m_collidingOnY(false){}
```

It simply initializes all of its data members to default values. Notice that out of all the members it sets to zero, the friction actually gets set up for the x axis to be 0.8. This is because we don't want the default behavior of the entity to be equal to that of a cow on ice, to put it frankly. Friction defines how much of the entity's velocity is lost to the environment. If it doesn't make too much sense now, don't worry. We're about to cover it in greater detail.

Here we have all of the methods for modifying data members of the entity base class:

```
void EntityBase::SetPosition(const float& l_x, const float& l_y){
    m_position = sf::Vector2f(l_x,l_y);
    UpdateAABB();
}
void EntityBase::SetPosition(const sf::Vector2f& l_pos){
    m_position = l_pos;
    UpdateAABB();
}
void EntityBase::SetSize(const float& l_x, const float& l_y){
    m_size = sf::Vector2f(l_x,l_y);
    UpdateAABB();
}
void EntityBase::SetState(const EntityState& l_state){
    if(m_state == EntityState::Dying){ return; }
    m_state = l_state;
}
```

As you can see, modifying either the position or size of an entity results in a call of the internal method `UpdateAABB`. Simply put, it's responsible for updating the position of the bounding box. More information on that is coming soon.

One interesting thing to note is in the `SetState` method. It does not allow the state to change if the current state is `Dying`. This is done in order to prevent some other event in the game to snap an entity out of death magically.

Now we have a more interesting chunk of code, responsible for moving an entity:

```
void EntityBase::Move(float l_x, float l_y){
    m_positionOld = m_position;
    m_position += sf::Vector2f(l_x,l_y);
    sf::Vector2u mapSize = m_entityManager->
        GetContext()->m_gameMap->GetMapSize();
    if(m_position.x < 0){
        m_position.x = 0;
    } else if(m_position.x > (mapSize.x + 1) * Sheet::Tile_Size){
        m_position.x = (mapSize.x + 1) * Sheet::Tile_Size;
    }

    if(m_position.y < 0){
        m_position.y = 0;
    } else if(m_position.y > (mapSize.y + 1) * Sheet::Tile_Size){
        m_position.y = (mapSize.y + 1) * Sheet::Tile_Size;
        SetState(EntityState::Dying);
    }

    UpdateAABB();
}
```

First, we copy the current position to another data member: `m_positionOld`. It's always good to keep track of this information, in case we need it later. Then, the position is adjusted by the offset provided through the arguments. The size of the map is obtained afterwards, in order to check the current position for being outside of the map. If it is on either axis, we simply reset its position to something that's at the very edge of the out-of-bounds area. In the case of the entity being outside of the map on the *y* axis, its state is set to `Dying`. After all of that, the bounding box is updated in order to reflect the changes to the position of the entity sprite.

Now let's work on adding to and managing the entity's velocity:

```
void EntityBase::AddVelocity(float l_x, float l_y){
    m_velocity += sf::Vector2f(l_x,l_y);
    if(abs(m_velocity.x) > m_maxVelocity.x){
        if(m_velocity.x < 0){ m_velocity.x = -m_maxVelocity.x; }
        else { m_velocity.x = m_maxVelocity.x; }
    }

    if(abs(m_velocity.y) > m_maxVelocity.y){
        if(m_velocity.y < 0){ m_velocity.y = -m_maxVelocity.y; }
        else { m_velocity.y = m_maxVelocity.y; }
    }
}
```

As you can see, it's fairly simple stuff. The velocity member is added to and then checked for being outside of the bounds of allowed maximum velocity. In the first check we're using absolute values, because velocity can be both positive and negative, which indicates the direction the entity's moving in. If the velocity is out of bounds, it gets reset to the maximum allowed value it can have.

Accelerating an entity, you could say, is as simple as adding one vector to another:

```
void EntityBase::Accelerate(float l_x, float l_y){
    m_acceleration += sf::Vector2f(l_x,l_y);
}
```

Applying friction is no more complex than managing our velocity:

```
void EntityBase::ApplyFriction(float l_x, float l_y){
    if(m_velocity.x != 0){
        if(abs(m_velocity.x) - abs(l_x) < 0){ m_velocity.x = 0; }
        else {
            if(m_velocity.x < 0){ m_velocity.x += l_x; }
            else { m_velocity.x -= l_x; }
        }
    }

    if(m_velocity.y != 0){
        if (abs(m_velocity.y) - abs(l_y) < 0){ m_velocity.y = 0; }
        else {
            if(m_velocity.y < 0){ m_velocity.y += l_y; }
            else { m_velocity.y -= l_y; }
        }
    }
}
```

It needs to check if the difference between the absolute values of both the velocity and the friction coefficient on that axis isn't less than zero, in order to prevent changing the direction of the entity's movement through friction, which would simply be weird. If it is less than zero, the velocity gets set back to zero. If it isn't, the velocity's sign is checked and friction in the proper direction is applied.

In order for an entity to not be a static part of the backdrop, it needs to be updated:

```
void EntityBase::Update(float l_dT){
    Map* map = m_entityManager->GetContext()->m_gameMap;
    float gravity = map->GetGravity();
    Accelerate(0,gravity);
    AddVelocity(m_acceleration.x * l_dT, m_acceleration.y * l_dT);
    SetAcceleration(0.0f, 0.0f);
}
```

```

sf::Vector2f frictionValue;
if(m_referenceTile){
    frictionValue = m_referenceTile->m_friction;
    if(m_referenceTile->m_deadly){ SetState(EntityState::Dying); }
} else if(map->GetDefaultTile()){
    frictionValue = map->GetDefaultTile()->m_friction;
} else {
    frictionValue = m_friction;
}

float friction_x = (m_speed.x * frictionValue.x) * l_dT;
float friction_y = (m_speed.y * frictionValue.y) * l_dT;
ApplyFriction(friction_x, friction_y);
sf::Vector2f deltaPos = m_velocity * l_dT;
Move(deltaPos.x, deltaPos.y);
m_collidingOnX = false;
m_collidingOnY = false;
CheckCollisions();
ResolveCollisions();
}

```

Quite a bit is happening here. Let's take it step by step. First, an instance of the game map is obtained through the shared context. It is then used to obtain the gravity of the map, which was loaded from the map file. The entity's acceleration is then increased by the gravity on the *y* axis. By using the `AddVelocity` method and passing in the acceleration multiplied by delta time, the velocity is adjusted and the acceleration is set back to zero. Next, we must obtain the friction coefficient that the velocity will be damped by. The `m_referenceTile` data member, if it's not set to `nullptr`, is used first, in order to obtain the friction from a tile the entity's standing on. If it is set to `nullptr`, the entity must be in mid-air, so the default tile from the map is obtained to grab the friction values that were loaded from the map file. If that, for whatever reason, is also not set up, we default to the value set in the `EntityBase`'s constructor.

Before we get to calculating friction, it's important to clarify that the `m_speed` data member is not set up or initialized in this class, aside from being set to a default value. The speed is how much an entity is accelerated when it's moving and it will be implemented in one of the derived classes of `EntityBase`.

If you recall from the constructor of this class, we set up the default friction to be 0.8f. That is not just an incredibly small value. We're using friction as a factor in order to determine how much of the entity's speed should be lost. Having said that, multiplying the speed by a friction coefficient and multiplying that by delta time yields us the velocity that is lost during this frame, which is then passed into the `ApplyFriction` method in order to manipulate the velocity.

Finally, the change in position, called `deltaPos` is calculated by multiplying the velocity by delta time, and is passed into the `Move` method to adjust the entity's position in the world. The flags for collisions on both axes get reset to false and the entity calls its own private members for first obtaining and then resolving collisions.

Let's take a look at the method responsible for updating the bounding box:

```
void EntityBase::UpdateAABB() {
    m_AABB = sf::FloatRect(m_position.x - (m_size.x / 2),
        m_position.y - m_size.y, m_size.x, m_size.y);
}
```

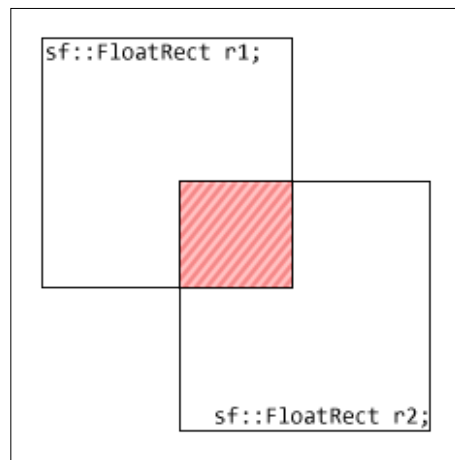
Because the origin of the bounding box is left at the top-left corner and the entity's position is set to (width / 2, height), accounting for that is necessary if we want to have accurate collisions. The rectangle that represents the bounding box is reset to match the new position of the sprite.

Entity-on-tile collisions

Before jumping into collision detection and resolution, let's revisit the method SFML provides to check if two rectangles are intersecting:

```
sf::FloatRect r1;
sf::FloatRect r2;
if(r1.intersects(r2)) { ... }
```

It doesn't matter which rectangle we check, the intersecting method will still return true if they are intersecting. However, this method does take in an optional second argument, which is a reference of a rectangle class that will be filled with the information about the intersection itself. Consider the following illustration:



We have two rectangles that are intersecting. The diagonal striped area represents the rectangle of intersection, which can be obtained by doing this:

```
...
sf::FloatRect intersection;
if(r1.intersects(r2, intersection)) { ... }
```

This is important to us, because an entity could be colliding with more than one tile at a time. Knowing the depth of a collision is also a crucial part of resolving it. With that in mind, let's define a structure to temporarily hold the collision information before it gets resolved:

```
struct CollisionElement{
    CollisionElement(float l_area, TileInfo* l_info,
        const sf::FloatRect& l_bounds):m_area(l_area),
        m_tile(l_info), m_tileBounds(l_bounds){}
    float m_area;
    TileInfo* m_tile;
    sf::FloatRect m_tileBounds;
};

using Collisions = std::vector<CollisionElement>;
```

First, we're creating a structure that holds a floating point number representing the area of collision, a rectangle that holds the boundary information of a tile the entity's colliding with, and a pointer to a `TileInfo` instance. You always want to resolve the biggest collisions first, and this information is going to help us do just that. The collision elements themselves are going to be stored in a vector this time.

Next, we need a function that can compare two elements of our custom container in order to sort it, the blueprint of which in the header file of the `EntityBase` class looks like this:

```
bool SortCollisions(const CollisionElement& l_1,
    const CollisionElement& l_2);
```

Implementing this function is incredibly easy. The vector container simply uses a Boolean check to determine which one of the two elements it's comparing is larger. We simply return true or false, based on which element is bigger. Because we're sorting our container by the area size, the comparison is done between the first elements of the first pairs:

```
bool SortCollisions(const CollisionElement& l_1,
    const CollisionElement& l_2)
{ return l_1.m_area > l_2.m_area; }
```

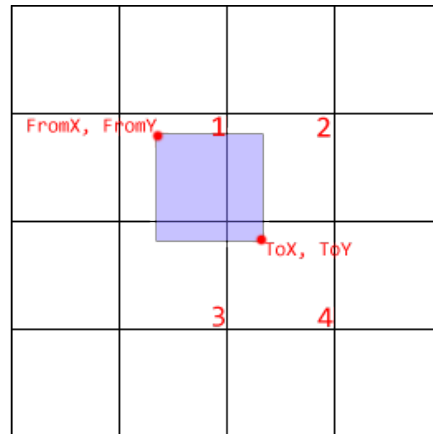
Now onto the interesting part, detecting the collisions:

```
void EntityBase::CheckCollisions(){
    Map* gameMap = m_entityManager->GetContext()->m_gameMap;
    unsigned int tileSize = gameMap->GetTileSize();
    int fromX = floor(m_AABB.left / tileSize);
    int toX = floor((m_AABB.left + m_AABB.width) / tileSize);
    int fromY = floor(m_AABB.top / tileSize);
    int toY = floor((m_AABB.top + m_AABB.height) / tileSize);

    for(int x = fromX; x <= toX; ++x){
        for(int y = fromY; y <= toY; ++y){
            Tile* tile = gameMap->GetTile(x,y);
            if (!tile){ continue; }
            sf::FloatRect tileBounds(x * tileSize, y * tileSize,
                tileSize,tileSize);
            sf::FloatRect intersection;
            m_AABB.intersects(tileBounds,intersection);
            float area = intersection.width * intersection.height;

            CollisionElement e(area, tile->m_properties, tileBounds);
            m_collisions.emplace_back(e);
            if(tile->m_warp && m_type == EntityType::Player){
                gameMap->LoadNext();
            }
        }
    }
}
```

We begin by using the coordinates and size of the bounding box to obtain the coordinates of tiles it is potentially intersecting. This is illustrated better in the following image:



The range of tile coordinates represented by the four integers is then fed into a double loop which checks if there is a tile occupying the space we're interested in. If a tile is returned from the `GetTile` method, the bounding box of the entity is definitely intersecting a tile, so a float rectangle that represents the bounding box of a tile is created. We also prepare another float rectangle to hold the data of the intersection and call the `intersects` method in order to obtain this information. The area of the intersection is calculated by multiplying its width and height, and the information about the collision is pushed into the collision container, along with a pointer to the `TileInfo` object that represents the type of tile the entity is colliding with.

The last thing we do before wrapping up this method is check if the current tile the entity is colliding with is a warp tile and if the entity is a player. If both of these conditions are met, the next map is loaded.

Now that a list of collisions for an entity has been obtained, resolving them is the next step:

```
void EntityBase::ResolveCollisions() {
    if (!m_collisions.empty()) {
        std::sort(m_collisions.begin(),
            m_collisions.end(), SortCollisions);
        Map* gameMap = m_entityManager->GetContext()->m_gameMap;
        unsigned int tileSize = gameMap->GetTileSize();
        for (auto &itr : m_collisions) {
            if (!m_AABB.intersects(itr.m_tileBounds)) { continue; }
        }
    }
}
```

```
float xDiff = (m_AABB.left + (m_AABB.width / 2)) -
    (itr.m_tileBounds.left + (itr.m_tileBounds.width / 2));
float yDiff = (m_AABB.top + (m_AABB.height / 2)) -
    (itr.m_tileBounds.top + (itr.m_tileBounds.height / 2));
float resolve = 0;
if(abs(xDiff) > abs(yDiff)){
    if(xDiff > 0){
        resolve = (itr.m_tileBounds.left + tileSize) -
            m_AABB.left;
    } else {
        resolve = -(m_AABB.left + m_AABB.width) -
            itr.m_tileBounds.left);
    }
    Move(resolve, 0);
    m_velocity.x = 0;
    m_collidingOnX = true;
} else {
    if(yDiff > 0){
        resolve = (itr.m_tileBounds.top + tileSize) -
            m_AABB.top;
    } else {
        resolve = - ((m_AABB.top + m_AABB.height) -
            itr.m_tileBounds.top);
    }
    Move(0, resolve);
    m_velocity.y = 0;
    if (m_collidingOnY){ continue; }
    m_referenceTile = itr.m_tile;
    m_collidingOnY = true;
}
}
m_collisions.clear();
}
if(!m_collidingOnY){ m_referenceTile = nullptr; }
}
```

First, we check if there are any collisions in the container. Sorting of all the elements happens next. The `std::sort` function is called and iterators to the beginning and end of the container are passed in, along with the name of the function that will do the comparisons between the elements.

The code proceeds to loop over all of the collisions stored in the container. There is another intersection check here between the bounding box of the entity and the tile. This is done because resolving a previous collision could have moved an entity in such a way that it is no longer colliding with the next tile in the container. If there still is a collision, distances from the center of the entity's bounding box to the center of the tile's bounding box are calculated. The first purpose these distances serve is illustrated in the next line, where their absolute values get compared. If the distance on the x axis is bigger than on the y axis, the resolution takes place on the x axis. Otherwise, it's resolved on the y axis.

The second purpose of the distance calculation is determining which side of the tile the entity is on. If the distance is positive, the entity is on the right side of the tile, so it gets moved in the positive x direction. Otherwise, it gets moved in the negative x direction. The *resolve* variable takes in the amount of penetration between the tile and the entity, which is different based on the axis and the side of the collision.

In the case of both axes, the entity is moved by calling its `Move` method and passing in the depth of penetration. Killing the entity's velocity on that axis is also important, in order to simulate the entity hitting a solid. Lastly, the flag for a collision on a specific axis is set to true.

If a collision is resolved on the y axis, in addition to all the same steps that are taken in a case of x axis collision resolution, we also check if the flag is set for a y axis collision. If it hasn't been set yet, we change the `m_referenceTile` data member to point to the tile type of the current tile the entity is colliding with, which is followed by that flag getting set to true in order to keep the reference unchanged until the next time collisions are checked. This little snippet of code gives any entity the ability to behave differently based on which tile it's standing on. For example, the entity can slide a lot more on ice tiles than on simple grass tiles, as illustrated here:



As the arrow points out, the friction coefficient of these tiles is different, which means we are in fact obtaining the information from the tiles directly below.

Entity storage and management

Without proper management, these entities are just random classes scattered about in your memory with no rhyme or reason. In order to produce a robust way to create interactions between entities, they need to be babysat by a manager class. Before we begin designing it, let's define some data types to contain the information we're going to be working with:

```
using EntityContainer = std::unordered_map<
    unsigned int, EntityBase*>;
using EntityFactory = std::unordered_map<
    EntityType, std::function<EntityBase*(void)>>;
using EnemyTypes = std::unordered_map<std::string, std::string>;
```

The EntityContainer type is, as the name suggests, a container of entities. It is once again powered by an unordered_map, which ties instances of entities to unsigned integers that serve as identifiers. The next type is a container of lambda functions that links entity types to code that can allocate memory and return instances of classes that inherit from the base entity class and serves as a factory. This behavior isn't new to us, so let's move on to defining the entity manager class:

```
class EntityManager{
public:
    EntityManager(SharedContext* l_context,
        unsigned int l_maxEntities);
    ~EntityManager();

    int Add(const EntityType& l_type,
        const std::string& l_name = "");
    EntityBase* Find(unsigned int l_id);
    EntityBase* Find(const std::string& l_name);
    void Remove(unsigned int l_id);

    void Update(float l_dT);
    void Draw();

    void Purge();

    SharedContext* GetContext();
private:
    template<class T>
```

```

void RegisterEntity(const EntityType& l_type){
    m_entityFactory[l_type] = [this]() -> EntityBase*
    {
        return new T(this);
    };
}

void ProcessRemovals();
void LoadEnemyTypes(const std::string& l_name);
void EntityCollisionCheck();

EntityContainer m_entities;
EnemyTypes m_enemyTypes;
EntityFactory m_entityFactory;
SharedContext* m_context;
unsigned int m_idCounter;
unsigned int m_maxEntities;

std::vector<unsigned int> m_entitiesToRemove;
};

```

Aside from the private template method for inserting lambda functions into the entity factory container, this looks like a relatively typical class. We have methods for updating and drawing entities, adding, finding and removing them and purging all of the data, as we tend to do. The presence of the private method called `ProcessRemovals` insists that we're using delayed removals of entities, much like we did in our state manager class. Let's take a closer look at how this class will operate by implementing it.

Implementing the entity manager

As always, a good place to start is the constructor:

```

EntityManager::EntityManager(SharedContext* l_context,
    unsigned int l_maxEntities):m_context(l_context),
    m_maxEntities(l_maxEntities), m_idCounter(0)
{
    LoadEnemyTypes("EnemyList.list");
    RegisterEntity<Player>(EntityType::Player);
    RegisterEntity<Enemy>(EntityType::Enemy);
}
EntityManager::~EntityManager(){ Purge(); }

```

Some of its data members are initialized through an initializer list. The `m_idCounter` variable will be used to keep track of the highest ID that was given to an entity. Next, a private method is invoked for loading pairs of enemy names and their character definition files, which will be explained a little later.

Lastly, two entity types are registered: player and enemy. We don't have their classes set up yet, but it's coming soon, so we may as well just register them now.

The destructor of an entity manager simply invokes the `Purge` method.

Adding a new entity to the game is done by passing in an entity type along with its name to the `Add` method of the entity manager:

```
int EntityManager::Add(const EntityType& l_type,
    const std::string& l_name)
{
    auto itr = m_entityFactory.find(l_type);
    if (itr == m_entityFactory.end()) { return -1; }
    EntityBase* entity = itr->second();
    entity->m_id = m_idCounter;
    if (l_name != "") { entity->m_name = l_name; }
    m_entities.emplace(m_idCounter, entity);
    if(l_type == EntityType::Enemy) {
        auto itr = m_enemyTypes.find(l_name);
        if(itr != m_enemyTypes.end()) {
            Enemy* enemy = (Enemy*)entity;
            enemy->Load(itr->second);
        }
    }

    ++m_idCounter;
    return m_idCounter - 1;
}
```

The entity factory container is searched for the type that was provided as an argument. If that type is registered, the lambda function is invoked to allocate dynamic memory for the entity and the memory address is caught by a pointer variable to the `EntityBase` class - `entity`. The newly created entity is then inserted into the entity container and its ID is set up by using the `m_idCounter` data member. If the user provides an argument for the entity name, it gets set up as well.

The entity type then gets checked. If it's an enemy, the enemy type container is searched in order to locate the path to a character definition file. If it's found, the entity is type-cast into an enemy instance and a `Load` method is called, to which the character file path is passed.

Lastly, the ID counter is incremented and the entity ID that was just used gets returned to signify success. If the method failed at any point, it will instead return *-1*, signifying a failure.

Having an entity manager is pointless if you can't obtain the entities. That's where the `Find` method comes in:

```
EntityBase* EntityManager::Find(const std::string& l_name){
    for(auto &itr : m_entities){
        if(itr.second->GetName() == l_name){
            return itr.second;
        }
    }
    return nullptr;
}
```

Our entity manager provides two versions of this method. The first version takes in an entity name and searches the container until an entity is found with that name, at which point it gets returned. The second version looks up entities based on a numerical identifier:

```
EntityBase* EntityManager::Find(unsigned int l_id){
    auto itr = m_entities.find(l_id);
    if (itr == m_entities.end()){ return nullptr; }
    return itr->second;
}
```

Because we map instances of entities to numerical values, this is easier, as we can simply call the `Find` method of our container in order to find the element we're looking for.

Now let's work on removing entities:

```
void EntityManager::Remove(unsigned int l_id){
    m_entitiesToRemove.emplace_back(l_id);
}
```

This is the public method that takes in an entity ID and inserts it into a container, which will be used later to remove entities.

Updating all entities can be achieved as follows:

```
void EntityManager::Update(float l_dT){
    for(auto &itr : m_entities){
        itr.second->Update(l_dT);
    }
}
```

```
    EntityCollisionCheck();  
    ProcessRemovals();  
}
```

The manager iterates through all of its elements and invokes their respective `Update` methods by passing in the delta time it receives as an argument. After all of the entities are updated, a private method `EntityCollisionCheck` is invoked in order to check for and resolve collisions between entities. Then, we process the entity removals that were added by the `Remove` method implemented previously.

Let's take a look at how we can draw all of these entities:

```
void EntityManager::Draw() {  
    sf::RenderWindow* wnd = m_context->m_wind->GetRenderWindow();  
    sf::FloatRect viewSpace = m_context->m_wind->GetViewSpace();  
  
    for(auto &itr : m_entities) {  
        if (!viewSpace.intersects(itr.second->m_AABB)) { continue; }  
        itr.second->Draw(wnd);  
    }  
}
```

After obtaining a pointer to the render window, we also get the view space of it in order to cull entities for efficiency reasons. Because both the view space and the bounding box of an entity are rectangles, we can simply check if they're intersecting in order to determine if an entity is within the view space, and if it is, it gets drawn.

The entity manager needs to have a way to dispatch of all of its resources. This is where the `Purge` method comes in:

```
void EntityManager::Purge() {  
    for (auto &itr : m_entities) {  
        delete itr.second;  
    }  
    m_entities.clear();  
    m_idCounter = 0;  
}
```

Entities get iterated over and their dynamic memory is de-allocated – regular as clockwork. Now to process the entities that need to be removed:

```
void EntityManager::ProcessRemovals() {  
    while(m_entitiesToRemove.begin() != m_entitiesToRemove.end()) {  
        unsigned int id = m_entitiesToRemove.back();  
        auto itr = m_entities.find(id);  
        if(itr != m_entities.end()) {
```

```

        std::cout << "Discarding entity: "
        << itr->second->GetId() << std::endl;
        delete itr->second;
        m_entities.erase(itr);
    }
    m_entitiesToRemove.pop_back();
}

```

As we're iterating over the container that holds the IDs of entities that need to be removed, the entity container is checked for the existence of every ID that was added. If an entity with the ID does in fact exist, its memory is de-allocated and the element is popped from the entity container.

Now for the interesting part – detecting entity-to-entity collisions:

```

void EntityManager::EntityCollisionCheck() {
    if (m_entities.empty()) { return; }
    for(auto itr = m_entities.begin();
        std::next(itr) != m_entities.end(); ++itr)
    {
        for(auto itr2 = std::next(itr);
            itr2 != m_entities.end(); ++itr2)
        {
            if(itr->first == itr2->first) { continue; }

            // Regular AABB bounding box collision.
            if(itr->second->m_AABB.intersects(itr2->second->m_AABB)) {
                itr->second->OnEntityCollision(itr2->second, false);
                itr2->second->OnEntityCollision(itr->second, false);
            }

            EntityType t1 = itr->second->GetType();
            EntityType t2 = itr2->second->GetType();
            if (t1 == EntityType::Player || t1 == EntityType::Enemy) {
                Character* c1 = (Character*)itr->second;
                if (c1->m_attackAABB.intersects(itr2->second->m_AABB)) {
                    c1->OnEntityCollision(itr2->second, true);
                }
            }

            if (t2 == EntityType::Player || t2 == EntityType::Enemy) {
                Character* c2 = (Character*)itr2->second;
                if (c2->m_attackAABB.intersects(itr->second->m_AABB)) {

```

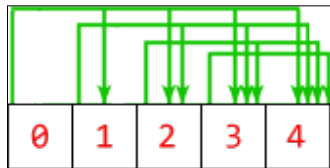
```
        c2->OnEntityCollision(itr->second, true);
    }
}
}
}
```

First, the way we're checking every entity against every other entity needs to be addressed. There are, of course, much better and more efficient ways to determine which entities to check without simply iterating over all of them, such as binary space partitioning. However, given the scope of our project, that would be overkill:

"Premature optimization is the root of all evil (or at least most of it) in programming."

– Donald Knuth

Having said that, we are going to be a bit smarter and not simply iterate over all of the entities twice. Because checking entity 0 against entity 1 is the same as checking entity 1 against 0, we can implement a much more efficient algorithm by using `std::next`, which creates an iterator that is one space ahead of the one fed to it, and use it in the second loop. This creates a check pattern that looks something like this:



That is about as much optimization as we need in the early stages of making a game.

When iterating over entities, the collision check method first makes sure that both iterators do not share the same entity ID, for some odd reason. Then, it's simply a matter of checking for intersections between the bounding boxes of the two entities we're interested in. If there is a collision, the methods for handling it are called in both instances, passing in the entity being collided with as an argument, along with false as the second argument, to let the entity know it's a simple AABB collision. What does that mean? Well, generally, there are going to be two types of collisions between entities: regular bounding box collisions and attack collisions. Children of the `EntityBase` class, mainly the `Character` instances, will have to keep another bounding box in order to perform attacks, as illustrated here:



Because this isn't terribly complicated to implement, we can continue implementing the entity manger until we implement the `Character` class shortly.

Since only the `Character` class and any class that inherits from it is going to have an attack bounding box, it's necessary to first check if we're dealing with a `Character` instance by verifying the entity type. If an entity is of the type `Enemy` or `Player`, the `OnEntityCollision` method of the `Character` instance is invoked and receives the entity it's colliding with, as well as a Boolean constant of `true` this time, as arguments, to indicate an attack collision.

We're mostly done. Let's write the method for loading different enemy types that can parse files like this:

```
|Name|CharFile|
Rat Rat.char
```

It's quite a simple format. Let's read it in:

```
void EntityManager::LoadEnemyTypes(const std::string& l_name) {
    std::ifstream file;
    ... // Opening the file.
    while(std::getline(file, line)) {
        if (line[0] == '|') { continue; }
        std::stringstream keystream(line);
        std::string name;
        std::string charFile;
        keystream >> name >> charFile;
        m_enemyTypes.emplace(name, charFile);
    }
    file.close();
}
```

There is nothing here you haven't seen before. The two string values get read in and stored in the enemy type container. This simple bit of code concludes our interest in the entity manager class.

Using entities to build characters

So far, we only have entities that define some abstract methods and provide the means of manipulating them, but nothing that can appear in the game world, be rendered, and walk around. At the same time, we don't want to re-implement all of that functionality all over again in the player or enemy classes, which means we need an intermediate-level abstract class: *Character*. This class will provide all of the functionality that is shared between all entities that need to move around the world and be rendered. Let's get on with designing:

```
class Character : public EntityBase{
    friend class EntityManager;
public:
    Character(EntityManager* l_entityMgr);
    virtual ~Character();
    void Move(const Direction& l_dir);
    void Jump();
    void Attack();
    void GetHurt(const int& l_damage);
    void Load(const std::string& l_path);
    virtual void OnEntityCollision(
        EntityBase* l_collider, bool l_attack) = 0;
    virtual void Update(float l_dT);
    void Draw(sf::RenderWindow* l_wind);
}
```

```
protected:
    void UpdateAttackAABB();
    void Animate();
    SpriteSheet m_spriteSheet;
    float m_jumpVelocity;
    int m_hitpoints;
    sf::FloatRect m_attackAABB;
    sf::Vector2f m_attackAABBOffset;
};
```

First, let's talk about the public methods. Moving, jumping, attacking, and receiving damage are the common actions of every character-entity in the game. The character also has to be loaded in order to provide it with the correct graphics and properties that differ between each enemy type and the player. All classes derived from it have to implement their own version of handling collisions with other entities. Also, the `Update` method of the character class is made to be virtual, which allows any class inheriting from this one to either define its own update method or extend the existing one.

All characters will be using the sprite sheet class that we designed previously in order to support animations.

Implementing the character class

You know the drill by now. Here's the constructor:

```
Character::Character(EntityManager* l_entityMgr)
    :EntityBase(l_entityMgr),
    m_spriteSheet(m_entityManager->GetContext()->m_textureManager),
    m_jumpVelocity(250), m_hitpoints(5)
{ m_name = "Character"; }
```

The sprite sheet is created and set up by passing a pointer to the texture manager in its constructor. We also have a data member called `m_jumpVelocity`, which specifies how far the player can jump. Lastly, we set some arbitrary value to the `m_hitpoints` variable, which represents how many times an entity can be hit before it dies.

Let's move on to the `Move` method:

```
void Character::Move(const Direction& l_dir){
    if (GetState() == EntityState::Dying){ return; }
    m_spriteSheet.SetDirection(l_dir);
    if (l_dir == Direction::Left){ Accelerate(-m_speed.x, 0); }
    else { Accelerate(m_speed.x, 0); }
```

```
        if (GetState() == EntityState::Idle) {
            SetState(EntityState::Walking);
        }
    }
```

Regardless of the entity's direction, the state of the entity is checked in order to make sure the entity isn't dying. If it isn't, the direction of the sprite sheet is set up and the character begins to accelerate on a relevant axis. Lastly, if the entity is currently in an idle state, it gets set to walking simply to play the walking animation:

```
void Character::Jump() {
    if (GetState() == EntityState::Dying ||
        GetState() == EntityState::Jumping ||
        GetState() == EntityState::Hurt)
    {
        return;
    }
    SetState(EntityState::Jumping);
    AddVelocity(0, -m_jumpVelocity);
}
```

A character should only be able to jump if it isn't dying, taking damage, or jumping already. When those conditions are met and the character is instructed to jump, its state is set to `Jumping` and it receives negative velocity on the y axis that makes it combat the gravity force and go up. The velocity has to be high enough in order to break the gravitational force of the level.

Attacking is fairly straightforward. Because the entity manager already does the collision checking for us, all that's left to do is set the state if an entity isn't dying, jumping, taking damage, or already attacking:

```
void Character::Attack() {
    if (GetState() == EntityState::Dying ||
        GetState() == EntityState::Jumping ||
        GetState() == EntityState::Hurt ||
        GetState() == EntityState::Attacking)
    {
        return;
    }
    SetState(EntityState::Attacking);
}
```

In order to bestow mortality onto our entities, they need to have a way to be hurt:

```
void Character::GetHurt(const int& l_damage){
    if (GetState() == EntityState::Dying ||
        GetState() == EntityState::Hurt)
    {
        return;
    }
    m_hitpoints = (m_hitpoints - l_damage > 0 ?
        m_hitpoints - l_damage : 0);
    if (m_hitpoints){ SetState(EntityState::Hurt); }
    else { SetState(EntityState::Dying); }
}
```

This method inflicts damage to the character if it isn't already taking damage or dying. The damage value is either subtracted from the hit-points or the hitpoints variable is set to 0 in order to keep it from reaching the negatives. If the entity still has lives after the subtraction, its state is set to HURT in order to play the proper animation. Otherwise, the entity is sentenced to death by the programmer.

As previously mentioned, we want to be able to load our characters in from files like this one (Player.char):

```
Name Player
Spritesheet Player.sheet
Hitpoints 5
BoundingBox 20 26
DamageBox -5 0 26 26
Speed 1024 128
JumpVelocity 250
MaxVelocity 200 1024
```

It contains all the basic bits and pieces of what makes up a character, like the sprite sheet handle and all of the other information discussed in earlier sections. The loading method for this type of file will not differ much from the ones we've already implemented:

```
void Character::Load(const std::string& l_path){
    std::ifstream file;
    ...
    while(std::getline(file,line)){
        ...
        std::string type;
        keystream >> type;
        if(type == "Name"){
```

```
        keystream >> m_name;
    } else if(type == "Spritesheet"){
        std::string path;
        keystream >> path;
        m_spriteSheet.LoadSheet("media/SpriteSheets/" + path);
    } else if(type == "Hitpoints"){
        keystream >> m_hitpoints;
    } else if(type == "BoundingBox"){
        sf::Vector2f boundingSize;
        keystream >> boundingSize.x >> boundingSize.y;
        SetSize(boundingSize.x, boundingSize.y);
    } else if(type == "DamageBox"){
        keystream >> m_attackAABBOffset.x >> m_attackAABBOffset.y
            >> m_attackAABB.width >> m_attackAABB.height;
    } else if(type == "Speed"){
        keystream >> m_speed.x >> m_speed.y;
    } else if(type == "JumpVelocity"){
        keystream >> m_jumpVelocity;
    } else if(type == "MaxVelocity"){
        keystream >> m_maxVelocity.x >> m_maxVelocity.y;
    } else {
        std::cout << "! Unknown type in character file: "
            << type << std::endl;
    }
}
file.close();
}
```

Aside from the sprite sheet having to call a load method, the rest is simply loading in data members from a string stream.

Just like the base entity and its bounding box, the character has to have a way to update the position of its attack area:

```
void Character::UpdateAttackAABB(){
    m_attackAABB.left =
        (m_spriteSheet.GetDirection() == Direction::Left ?
        (m_AABB.left - m_attackAABB.width) - m_attackAABBOffset.x
        : (m_AABB.left + m_AABB.width) + m_attackAABBOffset.x);
    m_attackAABB.top = m_AABB.top + m_attackAABBOffset.y;
}
```

One subtle difference here is that the attack bounding box uses the position of the entity's bounding box, not its sprite position. Also, the way it's positioned is different based on the direction an entity is facing, due to the fact that the bounding box's position represents its top-left corner.

Now for the method that will make the biggest difference, visually speaking:

```
void Character::Animate(){
    EntityState state = GetState();

    if(state == EntityState::Walking && m_spriteSheet.
        GetCurrentAnim()->GetName() != "Walk")
    {
        m_spriteSheet.SetAnimation("Walk",true,true);
    }
    else if(state == EntityState::Jumping && m_spriteSheet.
        GetCurrentAnim()->GetName() != "Jump")
    {
        m_spriteSheet.SetAnimation("Jump",true,false);
    }
    else if(state == EntityState::Attacking && m_spriteSheet.
        GetCurrentAnim()->GetName() != "Attack")
    {
        m_spriteSheet.SetAnimation("Attack",true,false);
    } else if(state == EntityState::Hurt && m_spriteSheet.
        GetCurrentAnim()->GetName() != "Hurt")
    {
        m_spriteSheet.SetAnimation("Hurt",true,false);
    }
    else if(state == EntityState::Dying && m_spriteSheet.
        GetCurrentAnim()->GetName() != "Death")
    {
        m_spriteSheet.SetAnimation("Death",true,false);
    }
    else if(state == EntityState::Idle && m_spriteSheet.
        GetCurrentAnim()->GetName() != "Idle")
    {
        m_spriteSheet.SetAnimation("Idle",true,true);
    }
}
```

All it does is simply check the current state and the current animation. If the current animation does not match the current state, it gets set to something else. Note the use of the third argument in the `SetAnimation` method, which is a Boolean constant and represents animation looping. Certain animations do not need to loop, like the attack or hurt animation. The fact that they do not loop and are stopped when they reach the end frame gives us a hook to manipulate what happens in the game, simply based on the progress of a certain animation. Case in point – the `Update` method:

```
void Character::Update(float l_dT){
    EntityBase::Update(l_dT);
    if(m_attackAABB.width != 0 && m_attackAABB.height != 0){
```

```
        UpdateAttackAABB();
    }
    if(GetState() != EntityState::Dying && GetState() !=
        EntityState::Attacking && GetState() != EntityState::Hurt)
    {
        if(abs(m_velocity.y) >= 0.001f){
            SetState(EntityState::Jumping);
        } else if(abs(m_velocity.x) >= 0.1f){
            SetState(EntityState::Walking);
        } else {
            SetState(EntityState::Idle);
        }
    }
    else if(GetState() == EntityState::Attacking ||
        GetState() == EntityState::Hurt)
    {
        if(!m_spriteSheet.GetCurrentAnim()->IsPlaying()){
            SetState(EntityState::Idle);
        }
    }
    else if(GetState() == EntityState::Dying){
        if(!m_spriteSheet.GetCurrentAnim()->IsPlaying()){
            m_entityManager->Remove(m_id);
        }
    }
}
Animate();
m_spriteSheet.Update(l_dT);
m_spriteSheet.SetSpritePosition(m_position);
}
```

First, we invoke the update method of the entity's base class, because the character's state depends on it. Then, we check if the width and height of the attack bounding box aren't still at 0, which are the default values for them. If they aren't, it means the attack bounding box has been set up and can be updated. The rest of the update method pretty much just handles state transitions. If the entity isn't dying, attacking something, or taking damage, its current state is going to be determined by its velocity. In order to accurately depict an entity falling, we have to make the velocity on y axis take precedence over everything else. If the entity has no vertical velocity, it's checked for horizontal velocity instead and sets the state to `Walking` if the velocity is higher than the specified minimum. Using small values instead of absolute zero takes care of problems with animations being jittery sometimes.

Because the attacking and taking damage states are not set to loop, the sprite sheet animation is checked in order to see if it is still playing. If it isn't, the state is switched back to idle. Lastly, if the entity is dying and the dying animation is finished playing, we call the `Remove` method of our entity manager in order to remove this entity from the world.

The `Animate` method is called near the end of the update in order to reflect the state changes that may have taken place. Also, this is where the sprite sheet gets updated and has its position set to match the position of the entity.

After all of that code, let's end on something really simple – the `Draw` method:

```
void Character::Draw(sf::RenderWindow* l_wind){
    m_spriteSheet.Draw(l_wind);
}
```

Since our sprite-sheet class takes care of drawing, all we need to do is pass a pointer of a render window to its `Draw` method.

Creating the player

Now we have a solid base for creating entities that are visually represented on screen. Let's put that to good use and finally build our player class by starting with the header:

```
class Player : public Character{
public:
    Player(EntityManager* l_entityMgr);
    ~Player();

    void OnEntityCollision(EntityBase* l_collider, bool l_attack);
    void React(EventDetails* l_details);
};
```

This is where things get easy. Because we essentially "outsourced" most of the common functionality to the base classes, all we're left with now is player-specific logic. Notice the `React` method. Judging by its argument list, it's obvious that we're going to be using it as a callback for handling player input. Before we do that, however, we must register this method as one:

```
Player::Player(EntityManager* l_entityMgr)
    : Character(l_entityMgr)
{
    Load("Player.char");
    m_type = EntityType::Player;

    EntityManager* events = m_entityManager->
        GetContext()->m_eventManager;
    events->AddCallback<Player>(StateType::Game,
        "Player_MoveLeft", &Player::React, this);
    events->AddCallback<Player>(StateType::Game,
```

```
        "Player_MoveRight", &Player::React, this);
events->AddCallback<Player>(StateType::Game,
        "Player_Jump", &Player::React, this);
events->AddCallback<Player>(StateType::Game,
        "Player_Attack", &Player::React, this);
}
```

All we're doing here is calling the `Load` method in order to set up the character values for our player and adding multiple callbacks to the same `React` method that will be used to process keyboard input. The type of the entity is also set to `Player`:

```
Player::~Player() {
    EventManager* events =
        m_entityManager->GetContext()->m_eventManager;
events->RemoveCallback(GAME, "Player_MoveLeft");
events->RemoveCallback(GAME, "Player_MoveRight");
events->RemoveCallback(GAME, "Player_Jump");
events->RemoveCallback(GAME, "Player_Attack");
}
```

The destructor, predictably enough, simply removes callbacks that we were using to move the player around.

The last method we are required to implement by the `Character` class is responsible for entity-on-entity collision:

```
void Player::OnEntityCollision(EntityBase* l_collider,
    bool l_attack)
{
    if (m_state == EntityState::Dying){ return; }
    if(l_attack){
        if (m_state != EntityState::Attacking){ return; }
        if (!m_spriteSheet.GetCurrentAnim()->IsInAction()){ return; }
        if (l_collider->GetType() != EntityType::Enemy &&
            l_collider->GetType() != EntityType::Player)
        {
            return;
        }
        Character* opponent = (Character*)l_collider;
        opponent->GetHurt(1);
        if(m_position.x > opponent->GetPosition().x){
            opponent->AddVelocity(-32,0);
        } else {
            opponent->AddVelocity(32,0);
        }
    }
}
```

```
    } else {  
        // Other behavior.  
    }  
}
```

This method, as you remember from the entity manager portion of this chapter, is invoked when something is colliding with this particular entity. In a case of collision, the other colliding entity is passed in as an argument to this method together with a flag to determine if the entity is colliding with your bounding box or your attack region.

First, we make sure the player entity isn't dying. Afterwards, we check if it's the attack region that is colliding with another entity. If it is and the player is in the attack state, we check if the attack animation in the sprite sheet is currently "in action." If the current frame is within range of the beginning and end frames when the action is supposed to happen, the last check is made to determine if the entity is either a player or an enemy. Finally, if it is one or the other, the opponent gets hit with a pre-determined damage value, and based on its position will have some velocity added to it for a knock-back effect. That's about as basic a game design as it gets.

Adding enemies

In order to keep our player from walking the world lonely and un-attacked, we must add enemies to the game. Once again, let's begin with the header file:

```
#pragma once  
#include "Character.h"  
  
class Enemy : public Character{  
public:  
    Enemy(EntityManager* l_entityMgr);  
    ~Enemy();  
  
    void OnEntityCollision(  
        EntityBase* l_collider, bool l_attack);  
    void Update(float l_dT);  
private:  
    sf::Vector2f m_destination;  
    bool m_hasDestination;  
};
```

It's the same basic idea here as it was in the player class. This time, however, the enemy class needs to specify its own version of the `Update` method. It also has two private data members, one of which is a destination vector. It is a very simple attempt at adding basic artificial intelligence to the game. All it will do is keep track of a destination position, which the `Update` method will randomize every now and then to simulate wandering entities. Let's implement this:

```
Enemy::Enemy(EntityManager* l_entityMgr)
    :Character(l_entityMgr, m_hasDestination(false))
{
    m_type = EntityType::Enemy;
}
Enemy::~Enemy() {}
```

The constructor simply initializes a few data members to their default values, while the destructor remains unused. So far, so good!

```
void Enemy::OnEntityCollision(EntityBase* l_collider,
    bool l_attack)
{
    if (m_state == EntityState::Dying){ return; }
    if (l_attack){ return; }
    if (l_collider->GetType() != EntityType::Player){ return; }
    Character* player = (Character*)l_collider;
    SetState(EntityState::Attacking);
    player->GetHurt(1);
    if(m_position.x > player->GetPosition().x){
        player->AddVelocity(-m_speed.x,0);
        m_spriteSheet.SetDirection(Direction::Left);
    } else {
        player->AddVelocity(m_speed.y,0);
        m_spriteSheet.SetDirection(Direction::Right);
    }
}
```

The entity collision method is fairly similar as well, except this time we make sure to act if the enemy's bounding box is colliding with another entity, not its attack region. Also, we ignore every single collision, unless it's colliding with a player entity, in which case the enemy's state is set to `Attacking` in order to display the attack animation. It inflicts damage of 1 point to the player and knocks them back just a little bit based on where the entity is. The sprite-sheet direction is also set based on the position of the enemy entity relative to what it's attacking.

Now, to update our enemy:

```
void Enemy::Update(float l_dT) {
    Character::Update(l_dT);

    if (m_hasDestination) {
        if (abs(m_destination.x - m_position.x) < 16) {
            m_hasDestination = false;
            return;
        }
        if (m_destination.x - m_position.x > 0) {
            Move(Direction::Right);
        } else { Move(Direction::Left); }
        if (m_collidingOnX) { m_hasDestination = false; }
        return;
    }
    int random = rand() % 1000 + 1;
    if (random != 1000) { return; }
    int newX = rand() % 65 + 0;
    if (rand() % 2) { newX = -newX; }
    m_destination.x = m_position.x + newX;
    if (m_destination.x < 0) { m_destination.x = 0; }
    m_hasDestination = true;
}
```

Because this depends on the functionality of the `Character` class, we invoke its update method first before doing anything. Then the most basic simulation of A.I. begins by first checking if the entity has a destination. If it does not, a random number is generated between 1 and 1000. It has a 1/1000 chance to have its destination set to be anywhere within 128 pixels of its current position. The direction is decided by another random number generation, except much smaller this time. The destination finally is set and gets checked for being outside the world boundaries.

If, on the other hand, the entity does have a destination, the distance between it and its current position is checked. If it is above 16, the appropriate method for moving in a specific direction is called, based on which direction the destination point is in. We must also check for horizontal collisions, because an enemy entity could easily be assigned a destination that's beyond a tile it cannot cross. If that happens, the destination is simply taken away.

With that done, we now have wandering entities and a player that can be moved around the world! The only thing left to do in order to actually bring these entities into the game now is to load them.

Loading entities from the map file

If you recall from the section of this chapter that dealt with the issue of creating a map class, we haven't finished implementing the loading method fully, because we had no entities yet. With that no longer being the case, let's take a look at extending it:

```
} else if(type == "PLAYER"){
    if (playerId != -1){ continue; }
    // Set up the player position here.
    playerId = entityMgr->Add(EntityType::Player);
    if (playerId < 0){ continue; }
    float playerX = 0; float playerY = 0;
    keystore >> playerX >> playerY;
    entityMgr->Find(playerId)->SetPosition(playerX,playerY);
    m_playerStart = sf::Vector2f(playerX, playerY);
} else if(type == "ENEMY"){
    std::string enemyName;
    keystore >> enemyName;
    int enemyId = entityMgr->Add(EntityType::Enemy, enemyName);
    if (enemyId < 0){ continue; }
    float enemyX = 0; float enemyY = 0;
    keystore >> enemyX >> enemyY;
    entityMgr->Find(enemyId)->SetPosition(enemyX, enemyY);
} ...
```

If the map encounters a `PLAYER` line, it simply attempts to add an entity of type `Player` and grabs its ID. If it's above or equal to 0, the entity creation was successful, meaning that we can read in the rest of the data from the map file, which happens to be the player position. After obtaining it, we set the player's position and make sure we keep track of the starting position in the map class itself too.

All of the above is true for the `ENEMY` line as well, except it also loads in the name of the entity, which is needed in order to load its character information from the file.

Now our game is capable of loading entities from the map files and thus putting them into the game world like so:



Final editions to our code base

In this last portion of the chapter, we will be covering small changes and additions/editions that have been made all over the code written in the previous chapters in order to make this possible, starting with the shared context, which is now moved into its own header file.

Changes to the shared context

Out of all of the extra classes we defined, some of them need to be accessible to the rest of the code-base. This is what the shared context structure looks like now:

```
class Map;
struct SharedContext{
    SharedContext():
        m_wind(nullptr),
        m_eventManager(nullptr),
        m_textureManager(nullptr),
        m_entityManager(nullptr),
```

```
m_gameMap(nullptr) {}

Window* m_wind;
EventManager* m_eventManager;
TextureManager* m_textureManager;
EntityManager* m_entityManager;
Map* m_gameMap;
DebugOverlay m_debugOverlay;
};
```

The last object in it is the debug overlay we briefly discussed while working on the base entity class, which helps us see what's going on in our game by providing overlay graphics for tiles that entities collide with, warp tiles, and spike tiles, giving us the visual representations of entity bounding boxes and so on. Because the debug code was not essential to this chapter, snippets of it did not get included here, but they're present in the code that comes with it.

Putting all the pieces together

Next, we need to put instances of the code we worked so hard on in the right places, starting with the entity manager class, which goes straight into the game class as a data member:

```
class Game{
public:
    ...
private:
    ...
    EntityManager m_entityManager;
};
```

The map class instance is kept around in the game state class:

```
class State_Game : public BaseState{
public:
    ...
private:
    ...
    Map* m_gameMap;
};
```

The main game state is also responsible for setting up its own view and zooming in just enough to make the game look more appealing and less prone to cause squinting, not to mention initializing and loading the map:

```
void State_Game::OnCreate() {
    ...
    sf::Vector2u size = m_stateMgr->GetContext()->
        m_wind->GetWindowSize();
    m_view.setSize(size.x,size.y);
    m_view.setCenter(size.x/2,size.y/2);
    m_view.zoom(0.6f);
    m_stateMgr->GetContext()->m_wind->
        GetRenderWindow()->setView(m_view);

    m_gameMap = new Map(m_stateMgr->GetContext(), this);
    m_gameMap->LoadMap("media/Maps/map1.map");
}
```

Because the map is dynamically allocated, it must be deleted in the `OnDestroy` method of the game state:

```
void State_Game::OnDestroy() {
    ...
    delete m_gameMap;
    m_gameMap = nullptr;
}
```

Now onto the final piece of this puzzle – the game state update method:

```
void State_Game::Update(const sf::Time& l_time) {
    SharedContext* context = m_stateMgr->GetContext();
    EntityBase* player = context->m_entityManager->Find("Player");
    if(!player) {
        std::cout << "Respawning player..." << std::endl;
        context->m_entityManager->Add(EntityType::Player, "Player");
        player = context->m_entityManager->Find("Player");
        player->SetPosition(m_gameMap->GetPlayerStart());
    } else {
        m_view.setCenter(player->GetPosition());
        context->m_wind->GetRenderWindow()->setView(m_view);
    }

    sf::FloatRect viewSpace = context->m_wind->GetViewSpace();
    if(viewSpace.left <= 0) {
```

```
m_view.setCenter(viewSpace.width / 2, m_view.getCenter().y);
context->m_wind->GetRenderWindow()->setView(m_view);
} else if (viewSpace.left + viewSpace.width >
(m_gameMap->GetMapSize().x + 1) * Sheet::Tile_Size)
{
m_view.setCenter(((m_gameMap->GetMapSize().x + 1) *
Sheet::Tile_Size) - (viewSpace.width / 2),
m_view.getCenter().y);
context->m_wind->GetRenderWindow()->setView(m_view);
}

m_gameMap->Update(l_time.asSeconds());
m_stateMgr->GetContext()->
m_entityManager->Update(l_time.asSeconds());
}
```

First, we determine if the player is still alive in the game by searching for them by name. If the player isn't found, they must've died, so a re-spawn is in order. A new player entity is created and the starting coordinates of the map are passed to its `SetPosition` method.

Now comes the part where we manage how the view is scrolling. If the player entity exists, we set the view's centre to match the exact player position and use the shared context to obtain the render window, which will be using the updated view. Now, we have an issue of the screen leaving the boundaries of the map, which can be resolved by checking the top-left corner of the view space. If it's below or equal to zero, we set the view's centre on the x axis to a position that would put its top-left corner at the very edge of the screen, in order to prevent scrolling infinitely to the left. If, however, the view is outside of the map in the opposite direction, the view centre's x coordinate is set up so that the right side of it is also at the very edge of the map's boundaries.

Finally, the game map, along with the entity manager, is updated right here, because we don't want the map updating or entities moving around if the current state is different.

Summary

Congratulations on making it past the halfway point of this book! All of the code that was written, the design decisions, accounting for efficiency, and trial and error has brought you to this point. While the game we built is fairly basic, its architecture is also quite robust and expandable, and that is no small feat. Although some things in it may not be perfect, you have also followed the golden rule of getting it working first, before refining it, and now you have quite a few game design patterns under your belt to start building more complex game applications, as well as a solid code-base to expand and improve.

With the conclusion of this chapter, the second project of the book is officially finished. We have solved some quite tricky problems, written thousands of lines of code, and broadened our understanding of the game development process beyond the stages of myopic, callow naïveté, but the real adventure is still ahead of us. We may not know where it will ultimately lead us, but one thing is for sure: now is not a time to stop. See you in the next chapter.